

08 — Experiment Tracking and MLOps Lite Plan

Interpretable Machine Learning for Discovering Hidden Regulatory Switches in Non-Coding DNA

Field	Value
Document ID	MLO-001
Version	1.0
Status	Active
Created	2026-06-07
Related docs	Technical Design , Modeling Roadmap , Roadmap

1. Philosophy

This project uses a **lightweight, file-based MLOps approach** appropriate for a small team working on Kaggle. No complex infrastructure (MLflow servers, Kubernetes, cloud pipelines) is required. The system should be:

- **Simple:** File-based logging that any team member can read and update.
- **Reproducible:** Every result can be re-derived from logged metadata.
- **Portable:** Works identically on Kaggle, local machines, and Google Colab.
- **Evolvable:** Can be upgraded to W&B or MLflow later without losing historical data.

2. Folder Conventions

2.1 Top-Level Structure



2.2 Rules

1. `data/raw/` is **read-only** after initial download. Never modify raw files.
2. `data/processed/` and `data/features/` are derived from raw data. They can be regenerated by re-running the pipeline.
3. `experiments/` is the single source of truth for all experiment outputs.
4. Every experiment gets its own subdirectory under `experiments/logs/`, `experiments/models/`, and `experiments/figures/`.
5. No experiment outputs in `notebooks/`. Notebooks save outputs to `experiments/`.

3. Experiment Naming Convention

3.1 Format

```
exp_{NNN}_{model}_{features}_{variant}
```

Component	Description	Examples
NNN	Three-digit sequential number	001, 002, 015
model	Model family short name	logistic, rf, xgb, cnn, pretrained
features	Feature set short name	kmer4, kmer456, kmer_gc, onehot, dnabert2
variant	Optional: distinguishes sub-experiments	gcmatch, flanking, k562, gm12878

3.2 Examples

```
exp_001_logistic_kmer4
exp_002_rf_kmer456
exp_003_xgb_kmer_gc_motif
exp_004_cnn_kmer_gc_motif_k562
exp_005_cnn_onehot
exp_006_cnn_onehot_k562
exp_007_cnn_onehot_gm12878
exp_008_pretrained_dnabert2_emb
exp_009_xgb_kmer_dnabert2_hybrid
```

3.3 Naming Registry

Maintain a running experiment registry in `experiments/REGISTRY.md`:

```
# Experiment Registry
|
| ID | Name | Phase | Status | Key result |
|----|-----|-----|-----|-----|
| 001 | exp_001_logistic_kmer4 | 1 | Complete | AUROC = 0.72 |
| 002 | exp_002_rf_kmer456 | 1 | Complete | AUROC = 0.81 |
| 003 | exp_003_xgb_kmer_gc_motif | 1 | Running | -
```

4. Notebook Hygiene

4.1 Notebook Header Template

Every notebook must begin with a metadata cell:

```
## Experiment Metadata
## ID: 001
## Name: exp_001_logistic_kmer4
## Phase: 1
## Status: Complete
## Key result: AUROC = 0.72
```

4.2 Notebook Structure Rules

Rule	Rationale
1. Header cell with metadata	Reproducibility; experiment identification
2. Imports cell	All imports at the top; explicit versions logged
3. Configuration cell	All hyperparameters defined in one place (dict or YAML)
4. Data loading cell	Load from versioned paths; print shapes and checksums
5. Modeling cells	Train, evaluate, interpret — in clearly separated sections
6. Logging cell	Save config, metrics, and artifacts to <code>experiments/</code>
7. Summary cell	Print key results; compare to previous experiments
8. Clear outputs before commit	Keep repository clean; outputs are in <code>experiments/</code>

4.3 Kaggle-Specific Notebook Rules

Rule	Detail
Self-contained	Kaggle notebooks must include or import all required code
Dataset references	Use Kaggle dataset paths (<code>/kaggle/input/...</code>)
Output saving	Save to <code>/kaggle/working/</code> (auto-persisted as notebook output)
Install dependencies	<code>!pip install</code> at top if needed; comment with version
GPU check	Include a cell that checks GPU availability and prints device info

5. Versioning Strategy

5.1 Data Versioning

What	Version format	Example
ENCODE download	<code>{source}_{release}_{download_date}</code>	<code>cCRE_v3_2026-06-07</code>
Reference genome	<code>{assembly}_{patch}</code>	<code>hg38_GRCh38.p14</code>
Processed dataset	Semantic version	<code>processed_v0.1.0</code>
Feature cache	<code>{feature_type}_{config_hash}</code>	<code>kmer_k456_v0.1.0</code>
Split definition	Semantic version	<code>chrom_holdout_v0.1.0</code>

5.2 Code Versioning

- **Git** for all source code and notebooks (with outputs cleared).
- **Branching strategy**: `main` (stable) + `dev` (active development) + feature branches.
- **Commit messages**: Conventional commits format (`feat:`, `fix:`, `data:`, `exp:`, `docs:`).
- **Tags**: Tag releases that correspond to major experiment milestones (e.g., `v0.1.0-baseline`).

5.3 Model Versioning

Model artifact	Location	Naming
Trained model	<code>experiments/models/{exp_name}/model.pkl</code>	One model per experiment directory
Model config	<code>experiments/logs/{exp_name}/config.yaml</code>	Hyperparameters that produced the model
Model metrics	<code>experiments/logs/{exp_name}/metrics.json</code>	Evaluation metrics

6. Metrics Logging

6.1 Metrics File Format

Each experiment saves a `metrics.json`:

```
{
  "experiment_name": "exp_002_rf_kmer456",
  "timestamp": "2025-05-15T14:30:05Z",
  "model_family": "random_forest",
  "features": "kmer_456",
  "dataset_version": "ccRE_v3_2020-06-07",
  "split_version": "chrom_holdout_v0.1.0",
  "random_seed": 42,
  "hyperparameters": {
    "n_estimators": 100,
    "max_depth": 20,
    "min_samples_split": 5,
    "class_weight": "balanced"
  },
  "metrics": {
    "test": {
      "auROC": 0.812,
      "auprc": 0.795,
      "accuracy": 0.763,
      "f1_macro": 0.758,
      "precision": 0.742,
      "recall": 0.742
    },
    "validation": {
      "auROC": 0.823,
      "auprc": 0.806,
      "accuracy": 0.772,
      "f1_macro": 0.768,
      "precision": 0.780,
      "recall": 0.772
    },
    "cv_fold": {
      "auROC_mean": 0.810,
      "auROC_std": 0.01
    }
  },
  "training_time_seconds": 34,
  "peak_memory_mb": 4200,
  "environment": {
    "platform": "kaggle",
    "gpu": "v1a",
    "python_version": "3.10.10",
    "sklearn_version": "1.3.2"
  }
}
```

6.2 Configuration File Format

Each experiment saves a `config.yaml`:

```
experiment_name: exp_002_rf_kmer456
timestamp: 2025-05-15T14:30:05Z
model_family: random_forest
features: kmer_456
dataset_version: ccRE_v3_2020-06-07
split_version: chrom_holdout_v0.1.0
random_seed: 42
hyperparameters:
  n_estimators: 100
  max_depth: 20
  min_samples_split: 5
  class_weight: balanced
metrics:
  test:
    auROC: 0.812
    auprc: 0.795
    accuracy: 0.763
    f1_macro: 0.758
    precision: 0.742
    recall: 0.742
  validation:
    auROC: 0.823
    auprc: 0.806
    accuracy: 0.772
    f1_macro: 0.768
    precision: 0.780
    recall: 0.772
  cv_fold:
    auROC_mean: 0.810
    auROC_std: 0.01
training_time_seconds: 34
peak_memory_mb: 4200
environment:
  platform: kaggle
  gpu: v1a
  python_version: 3.10.10
  sklearn_version: 1.3.2
```

7. Model Artifact Storage

7.1 What to Save

Artifact	Format	Always save?
Trained model	<code>.pkl</code> (scikit-learn), <code>.pt</code> (PyTorch), <code>.json</code> (XGBoost)	Yes
Hyperparameters	<code>config.yaml</code>	Yes
Evaluation metrics	<code>metrics.json</code>	Yes
Feature importance	<code>feature_importance.csv</code>	Yes
Confusion matrix plot	<code>confusion_matrix.png</code>	Yes
ROC curve plot	<code>roc_curve.png</code>	Yes
SHAP summary plot	<code>shap_summary.png</code>	If SHAP was computed
Saliency maps	<code>saliency_sample_{N}.png</code>	If saliency was computed
Training logs (loss curves)	<code>training_log.csv</code>	For neural models
Experiment notes	<code>notes.md</code>	Recommended

7.2 What NOT to Save to Git

Artifact	Reason	Alternative
Raw data files	Too large	Document download procedure; cache on Kaggle
Feature matrices (.npz)	Derived; can be recomputed	Document computation procedure
Trained model binaries (large)	Can be 100 MB+	Store in Kaggle output or cloud storage
Reference genome FASTA	3+ GB	Document download source

7.3 .gitignore Template

```
.data
data/raw/
data/reference/
data/processed/
data/features/
|
| Large model artifact
experiments/models/**/*_pkl
experiments/models/**/*_pt
experiments/models/**/*_json
|
| Notebook output
notebooks/**/.ipynb_checkpoints/
|
| Python
__pycache__
*.py
*.env
|
| OS
__pycache__
thumbs.db
```

8. Reproducibility Checklist

Before declaring any experiment "complete," verify:

#	Check	How
1	Random seed is set and logged	Check <code>config.yaml</code> → <code>random_seed: 42</code>
2	Data version is logged	Check <code>config.yaml</code> → <code>dataset_version</code>
3	Split version is logged	Check <code>config.yaml</code> → <code>split.version</code>
4	All hyperparameters are logged	Compare <code>config.yaml</code> to code
5	Software versions are logged	Check <code>metrics.json</code> → <code>environment</code>
6	Metrics are saved	Check <code>metrics.json</code> exists and is non-empty
7	Feature computation is deterministic	Re-run feature extraction; verify checksums match
8	Model can be loaded and re-evaluated	Load saved model; predict on test set; verify metrics match
9	Notebook runs top-to-bottom	Clear outputs and re-run; verify success
10	No hardcoded absolute paths (except Kaggle <code>/kaggle/input/</code>)	Grep for local paths

Reproducibility Verification Command

```
# Quick verification script
import json
import hashlib

def verify_experiment(exp_dir):
    """Check that an experiment is reproducible."""
    checks = []

    # Check config exists
    config_path = f"{exp_dir}/config.yaml"
    checks.append(("config exists", os.path.exists(config_path)))

    # Check metrics exist
    metrics_path = f"{exp_dir}/metrics.json"
    checks.append(("metrics exist", os.path.exists(metrics_path)))

    # Check metrics are valid JSON
    try:
        with open(metrics_path) as f:
            metrics = json.load(f)
            checks.append(("metrics valid JSON", True))
            checks.append(("has random_seed", "random_seed" in metrics))
            checks.append(("has dataset_version", "dataset_version" in metrics))
            checks.append(("has split.version", "split.version" in metrics))
    except:
        checks.append(("metrics valid JSON", False))

    for name, passed in checks:
        status = "✓" if passed else "X"
        print(f"{status} {name}")

    return all(passed for _, passed in checks)
```

9. Upgrading to External Tracking (Optional, Later)

9.1 When to Upgrade

Stage	Recommendation
1. Experiment setup and data collection	Log all data sources
2. Feature engineering and model training	Log all hyperparameters
3. Model evaluation and deployment	Log all metrics

9.2 Migration Path

The file-based logging format is designed to be easily importable:

```
# Example: migrate to wandb
import wandb
import json
import yaml

for exp_dir in sorted(glob("experiments/lops/exp_*")):
    config = yaml.safe_load(open(f"{exp_dir}/config.yaml"))
    metrics = json.load(open(f"{exp_dir}/metrics.json"))
    run = wandb.init(project="regulatory-switch", name=config["experiment_name"], config=config)
    run.log(metrics["metrics"][0]["test"])
    run.finish()
```

End of Document 08 — Experiment Tracking and MLOps Lite Plan

Next: [09 — Risk Register](#)